Lecture 19: Floating Point Instructions

CMPS 221 – Computer Organization and Design

Adapted by Izzat El Hajj from textbook slides for Computer Organization and Design, by David Patterson and John Hennessy, Morgan Kaufmann.

FP Co-Processor

- FP hardware is coprocessor 1
  - Adjunct processor that extends the ISA
- Separate FP registers
  - 32 single-precision: $f0, $f1, … $f31
  - Paired for double-precision: $f0/$f1, $f2/$f3, …
    - Use even-numbered register to address the pair
- FP instructions operate only on FP registers
  - Programs generally don’t do integer ops on FP data, or vice versa
  - More registers with minimal code-size impact

FP Register Convention

- $f0-$f3: return values
- $f4-$f11,$f16-$f19: temporaries
- $f12-$f15: arguments
- $f20-$f31: saved registers

FP Instructions in MIPS

- Single-precision arithmetic
  - `add.s`, `sub.s`, `mul.s`, div.s
    - e.g., `add.s $f0, $f1, $f6`
- Double-precision arithmetic
  - `add.d`, `sub.d`, `mul.d`, `div.d`
    - e.g., `mul.d $f4, $f4, $f6`
- FP load and store instructions
  - `lwc1`, `ldc1`, `swc1`, `sdc1`
    - e.g., `ldc1 $f8, 32($sp)`
    - Note: base registers are integer registers

FP Instructions in MIPS

- Single- and double-precision comparison
  - `c.`*xx*`.s`, `c.`*xx*`.d` (*xx* is `eq`, `lt`, `le`, …)
    - e.g. `c.lt.s $f3, $f4`
  - Sets or clears FP condition-code bit
- Branch on FP condition code true or false
  - `bc1t`, `bc1f`
    - e.g., `bc1t TargetLabel`
- No immediate instructions
  - Put constants in global memory and load them when needed (use $gp register)

FP Example: °F to °C

- C code:

```
float f2c (float fahr) {
    return ((5.0f/9.0f)*(fahr - 32.0f));
}
```

  - `fahr` : $f12, result : $f0
  - 5.0, 9.0, 32.0 in global memory at offsets 36, 40, 44

- Compiled MIPS code:

```
f2c: lwc1  $f16, 36($gp)
     lwc1  $f17, 40($gp)
     div.s $f16, $f16, $f17
     lwc1  $f17, 44($gp)
     sub.s $f17, $f12, $f17
     mul.s $f0,  $f16, $f17
     jr    $ra
```

Example: Matrix Multiplication

- X = X + Y × Z
  - All 32 × 32 matrices, 64-bit double-precision elements
- C code:

```
for (i = 0; i != 32; i = i + 1)
  for (j = 0; j != 32; j = j + 1)
    for (k = 0; k != 32; k = k + 1)
      x[i][j] = x[i][j] + y[i][k] * z[k][j];
```

  - Loop indices: `i` : `$s0`, `j` : `$s1`, k : `$s2`
  - Array base addresses: `x` : `$s3`, `y` : `$s4`, `z` : `$s5`
    - Arrays have 32 x 32 doubles (8 bytes)

Example: Matrix Multiplication

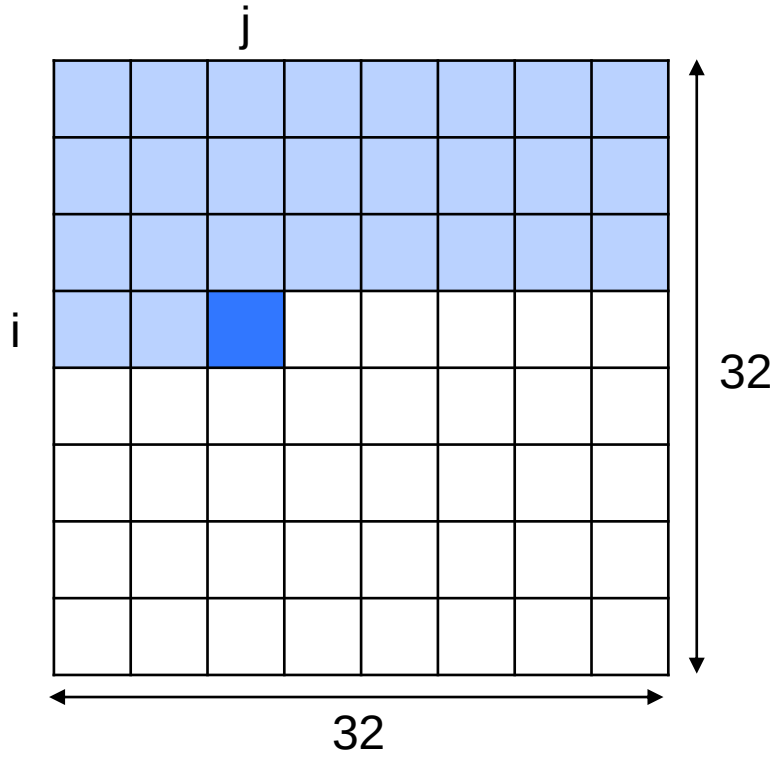
```
for (i = 0; i != 32; i = i + 1)
  for (j = 0; j != 32; j = j + 1)
    for (k = 0; k != 32; k = k + 1)
      x[i][j] = x[i][j] + y[i][k] * z[k][j];
```

i : $s0, j : $s1, k : $s2
x : $s3, y : $s4, z : $s5

```
        addi $s0, $zero, 0      # i = 0
L0:     addi $t0, $zero, 32
        beq  $s0, $t0, L0END    # i != 32
        addi $s1, $zero, 0      # j = 0
L1:     addi $t0, $zero, 32
        beq  $s1, $t0, L1END    # j != 32
        addi $s2, $zero, 0      # k = 0
L2:     addi $t0, $zero, 32
        beq  $s2, $t0, L2END    # k != 32
        ...                     # x[i][j] = x[i][j] + y[i][k] * z[k][j] (next slide)
        addi $s2, $s2, 1        # k = k + 1
        j    L2
L2END:  addi $s1, $s1, 1        # j = j + 1
        j L1
L1END:  addi $s0, $s0, 1        # i = i + 1
        j L0
L0END:  ...
```

Array Address Calculation

- How to calculate the address of `x[i][j]`?



Index of `x[i][j]`:

```
i*32 + j
```

Address of `x[i][j]`:
(recall: each element is 8 bytes)

```
x + (i*32 + j)*8
```

Example: Matrix Multiplication

```
x[i][j] = x[i][j] + y[i][k] * z[k][j];
```

i : $s0, j : $s1, k : $s2
x : $s3, y : $s4, z : $s5

```
sll  $t0, $s0, 5        # $t0 = i*32
add  $t0, $t0, $s1      # $t0 = i*32 + j
sll  $t0, $t0, 3        # $t0 = (i*32 + j)*8
add  $t0, $s3, $t0      # $t0 = x + (i*32 + j)*8
ldc1 $f16, 0($t0)       # $f16 = x[i][j]
sll  $t0, $s0, 5        # $t0 = i*32
add  $t0, $t0, $s2      # $t0 = i*32 + k
sll  $t0, $t0, 3        # $t0 = (i*32 + k)*8
add  $t0, $s4, $t0      # $t0 = y + (i*32 + k)*8
ldc1 $f18, 0($t0)       # $f18 = y[i][k]
sll  $t0, $s2, 5        # $t0 = k*32
add  $t0, $t0, $s1      # $t0 = k*32 + j
sll  $t0, $t0, 3        # $t0 = (k*32 + j)*8
add  $t0, $s5, $t0      # $t0 = z + (k*32 + j)*8
ldc1 $f20, 0($t0)       # $f20 = z[k][j]
mul.d $f18, $f18, $f20  # $f18 = y[i][k]*z[k][j]
add.d $f16, $f16, $f18  # $f16 = x[i][j] + y[i][k]*z[k][j]
sll  $t0, $s0, 5        # $t0 = i*32
add  $t0, $t0, $s1      # $t0 = i*32 + j
sll  $t0, $t0, 3        # $t0 = (i*32 + j)*8
add  $t0, $s3, $t0      # $t0 = x + (i*32 + j)*8
sdc1 $f16, 0($t0)       # x[i][j] = $f16
```

Textbook Sections

- The content in these slides corresponds to:
  - Textbook:
    - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
  - Sections:
    - 3.5